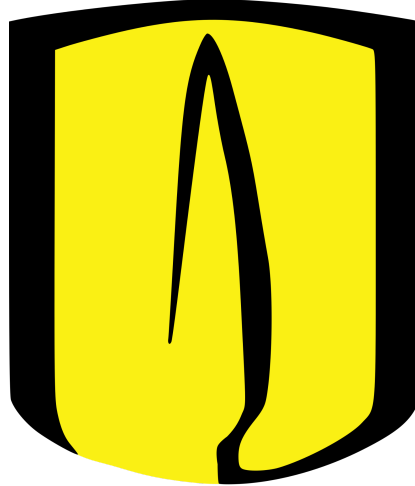


**UNIVERSIDAD DE LOS ANDES
DEPARTAMENTO DE INGENIERÍA DE
SISTEMAS Y COMPUTACIÓN**



**LABORATORIO 3: ANÁLISIS CAPA DE TRANSPORTE Y
SOCKETS**

**ISIS 2311 – REDES Y SERVICIOS DE
COMUNICACIONES**

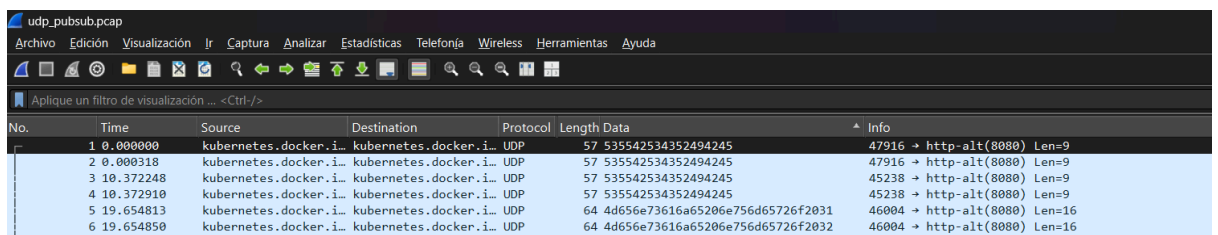
**Andrea Pinto
Nathalia Quiroga**

**Grupo 11
Sebastian Vargas Otero - 202323722
Pablo Ramirez Triana - 202321722
Yefran Cespedes - 202316255**

Contenido

1. Análisis UDP.....	2
2. Análisis TCP.....	4
3. Tabla comparativa:.....	6
4. Análisis de las Preguntas Planteadas.....	6

1. Análisis UDP



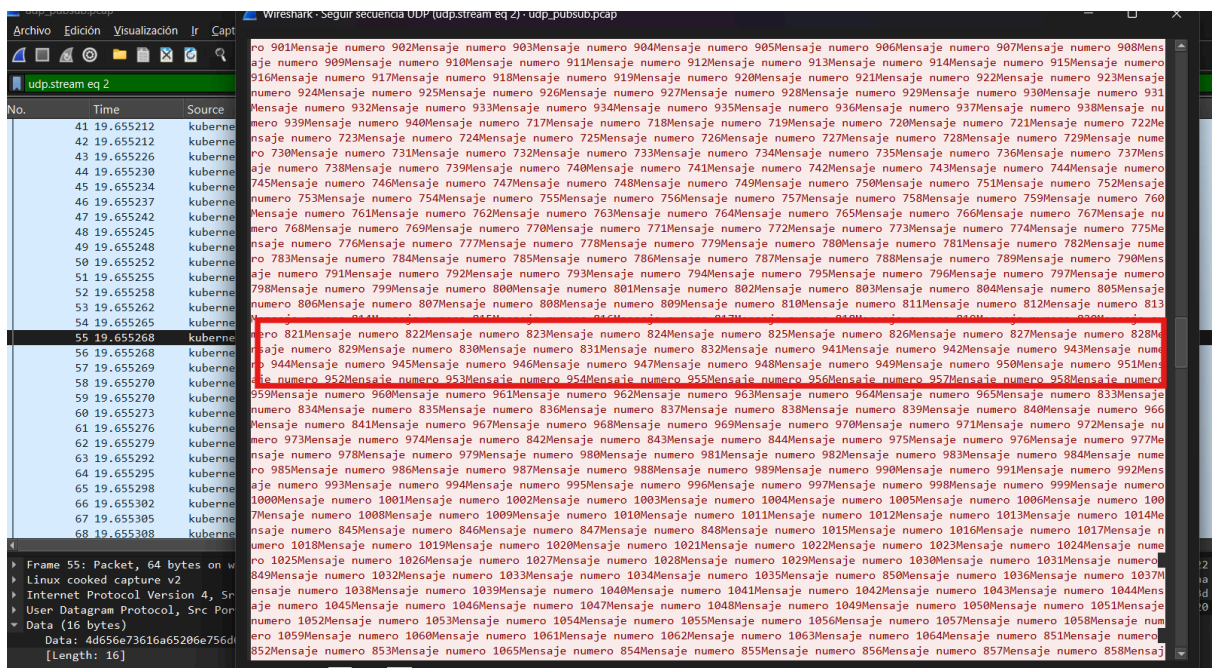
Wireshark capture showing UDP traffic from a Kubernetes pod to a service. The capture is filtered by 'udp.pubsucc.pcap'. The table below shows the first six packets, all of which are UDP messages from the pod to the service.

No.	Time	Source	Destination	Protocol	Length	Data	Info
1	0.000000	kubernetes.docker.i...	kubernetes.docker.i...	UDP	57	535542534352494245	47916 → http-alt(8080) Len=9
2	0.000318	kubernetes.docker.i...	kubernetes.docker.i...	UDP	57	535542534352494245	47916 → http-alt(8080) Len=9
3	10.372248	kubernetes.docker.i...	kubernetes.docker.i...	UDP	57	535542534352494245	45238 → http-alt(8080) Len=9
4	10.372910	kubernetes.docker.i...	kubernetes.docker.i...	UDP	57	535542534352494245	45238 → http-alt(8080) Len=9
5	19.654813	kubernetes.docker.i...	kubernetes.docker.i...	UDP	64	4d656e73616a65206e756d65726f2031	46004 → http-alt(8080) Len=16
6	19.654850	kubernetes.docker.i...	kubernetes.docker.i...	UDP	64	4d656e73616a65206e756d65726f2032	46004 → http-alt(8080) Len=16

Ilustración 1. Manejo de conexión UDP

Captura de los primeros paquetes enviados por UDP que presentan ausencia de Handshake

Desde el paquete 1 se están enviando datos de aplicación, los de longitud 9 consisten al mensaje de registro enviado por los suscriptores hacia el broker, posteriormente son mensajes del publicador, como no hay paquetes de sincronización confirma que UDP es un protocolo no orientado a la conexión stateless.



Wireshark capture showing a long sequence of UDP messages. The capture is filtered by 'udp.pubsucc.pcap'. The table below shows the first 68 packets, all of which are UDP messages from the pod to the service. The messages are numbered sequentially from 1 to 68.

No.	Time	Source
41	19.655212	kuberne...
42	19.655212	kuberne...
43	19.655226	kuberne...
44	19.655230	kuberne...
45	19.655234	kuberne...
46	19.655237	kuberne...
47	19.655242	kuberne...
48	19.655245	kuberne...
49	19.655248	kuberne...
50	19.655252	kuberne...
51	19.655255	kuberne...
52	19.655258	kuberne...
53	19.655262	kuberne...
54	19.655265	kuberne...
55	19.655268	kuberne...
56	19.655268	kuberne...
57	19.655269	kuberne...
58	19.655270	kuberne...
59	19.655270	kuberne...
60	19.655273	kuberne...
61	19.655276	kuberne...
62	19.655279	kuberne...
63	19.655292	kuberne...
64	19.655295	kuberne...
65	19.655298	kuberne...
66	19.655302	kuberne...
67	19.655305	kuberne...
68	19.655308	kuberne...

```
tcpdump: listening on any, link-type LINUX_SLL2 (Linux cooked v2), snapshot length 262144 bytes
^C72412 packets captured
151020 packets received by filter
78608 packets dropped by kernel
```

Ilustración 2. mensajes perdidos

Evidencia de pérdida masiva de mensajes y saturación de buffer por prueba de estrés

La imagen inferior presenta el resultado de la herramienta de captura de red (tcpdump) tras someter al sistema a una prueba de estrés enviando una ráfaga a alta velocidad. Se observa que la interfaz procesó un total de 151.020 paquetes, pero el kernel de Linux se vio obligado a descartar (drop) 78.608 de ellos. Este descarte masivo ocurre porque el buffer del socket receptor se saturó por completo. Al carecer UDP de mecanismos de control de flujo, el publicador transmitió a una velocidad superior a la capacidad de procesamiento del Broker, provocando el colapso del buffer.

La captura superior de Wireshark corrobora este evento a nivel de aplicación mediante el seguimiento del flujo de datos (UDP Stream). En el área resaltada en rojo se evidencia una clara ruptura en la secuencia lógica de los identificadores generados por el publicador (se observa un salto abrupto desde el "Mensaje número 832" directamente al "Mensaje número 941"). Esta discontinuidad demuestra empíricamente que los datagramas descartados por el kernel se pierden de manera definitiva, ya que el protocolo UDP, al no utilizar acuses de recibo (ACKs), no notifica la pérdida al emisor ni realiza ningún intento de retransmisión.

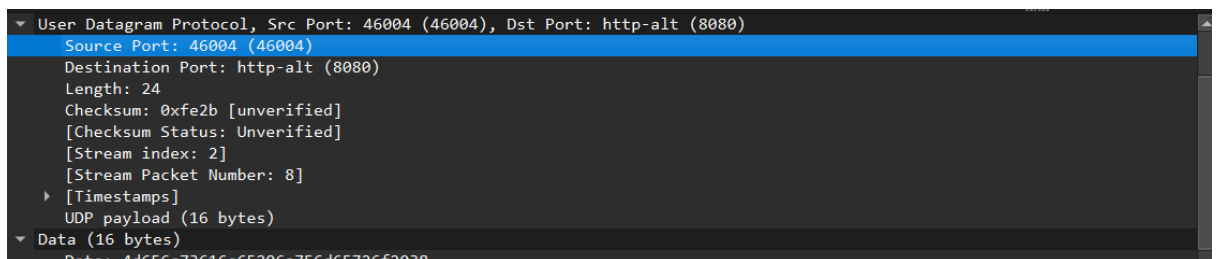


Ilustración 3. Carencia de control de flujo, ordenamiento nativo y bajo overhead

Análisis estructural de la cabecera UDP

Se observa la estructura interna de un datagrama UDP enviado desde el publicador hacia el Broker en el puerto de destino 8080 donde se comprueba que la cabecera de UDP consta de puerto de origen, puerto de destino, longitud total y checksum, donde cada paquete es de 24 bytes y por el udp payload ocupa 16 bytes, osea que los 8 bytes es el tamaño mínimo y estático de la cabecera. Tampoco hay variables de control como número de secuencia, el Acknowledgment o el tamaño de la ventana, udp no garantiza el orden de entrega ni provee mecanismos de control de flujo para evitar la saturación del nodo receptor.

- En UDP: ¿qué evidencias hay de pérdida o desorden en los mensajes?

La evidencia principal se obtuvo al realizar una prueba de estrés enviando una ráfaga masiva de datagramas (50,000 mensajes). Al revisar la captura de Wireshark y analizar el contenido del payload, se observaron saltos evidentes en la numeración interna de los mensajes generados por la aplicación. Esto ocurre porque UDP es un protocolo best-effort; al carecer de control de flujo, el publicador saturó la capacidad de procesamiento y el buffer del socket del Broker. El sistema operativo descartó miles de paquetes (comprobado a través de las estadísticas de tcpdump) y, al no existir mecanismos de acuse de recibo ni retransmisiones, esos mensajes se perdieron de forma definitiva.

2. Análisis TCP

2.1. Manejo de Conexión

La captura evidencia el ciclo completo de vida de cada conexión TCP. Los paquetes 249–252 muestran el Three-Way Handshake para el subscriber en el puerto 5002: SYN, SYN-ACK, ACK, seguido del [TCP Window Update] que ajusta el buffer. Los paquetes 173–180 al inicio de la captura muestran el cierre con [FIN, ACK], [ACK], [FIN, ACK], [ACK] del four-way handshake de una conexión anterior. Esto demuestra que TCP es orientado a conexión, esto quiere decir que no hay un solo byte de datos sin que primero exista un estado ESTABLISHED, y cada cierre es negociado explícitamente en lugar de simplemente dejar de enviar.

No.	Time	Source	Destination	Protocol	Length	Info
173	38.743195	Pablos-MacBook-Air...	Pablos-MacBook-Air...	TCP	56	rfe(5002) → 63546 [FIN, ACK] Seq=1 Ack=1 Win=6380 Len=0 TSval=2147004075 TSecr=437798786
174	38.743220	Pablos-MacBook-Air...	Pablos-MacBook-Air...	TCP	56	rfe(5002) → 63544 [FIN, ACK] Seq=1 Ack=1 Win=6380 Len=0 TSval=1313042771 TSecr=2350099322
175	38.743275	Pablos-MacBook-Air...	Pablos-MacBook-Air...	TCP	56	63546 → rfe(5002) [ACK] Seq=1 Ack=2 Win=6380 Len=0 TSval=438352891 TSecr=2147004075
176	38.743295	Pablos-MacBook-Air...	Pablos-MacBook-Air...	TCP	56	63544 → rfe(5002) [ACK] Seq=1 Ack=2 Win=6380 Len=0 TSval=2350653427 TSecr=1313042771
177	38.744064	Pablos-MacBook-Air...	Pablos-MacBook-Air...	TCP	56	63546 → rfe(5002) [FIN, ACK] Seq=1 Ack=2 Win=6380 Len=0 TSval=438352892 TSecr=2147004075
178	38.744117	Pablos-MacBook-Air...	Pablos-MacBook-Air...	TCP	56	rfe(5002) → 63546 [ACK] Seq=2 Ack=2 Win=6380 Len=0 TSval=2147004076 TSecr=438352892
179	38.744401	Pablos-MacBook-Air...	Pablos-MacBook-Air...	TCP	56	63544 → rfe(5002) [FIN, ACK] Seq=1 Ack=2 Win=6380 Len=0 TSval=2350653428 TSecr=1313042771
180	38.744440	Pablos-MacBook-Air...	Pablos-MacBook-Air...	TCP	56	rfe(5002) → 63544 [ACK] Seq=2 Ack=2 Win=6380 Len=0 TSval=1313042773 TSecr=2350653428
249	71.513799	Pablos-MacBook-Air...	Pablos-MacBook-Air...	TCP	68	63620 → rfe(5002) [SYN] Seq=0 Win=65535 Len=0 MSS=16344 WS=64 TSval=3010298339 TSecr=0 SACK_PERM
250	71.513881	Pablos-MacBook-Air...	Pablos-MacBook-Air...	TCP	68	rfe(5002) → 63620 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=16344 WS=64 TSval=2924905745 TSecr=30102
251	71.513901	Pablos-MacBook-Air...	Pablos-MacBook-Air...	TCP	56	63620 → rfe(5002) [ACK] Seq=1 Ack=1 Win=408320 Len=0 TSval=3010298339 TSecr=2924905745
252	71.513914	Pablos-MacBook-Air...	Pablos-MacBook-Air...	TCP	56	[TCP Window Update] rfe(5002) → 63620 [ACK] Seq=1 Ack=1 Win=408320 Len=0 TSval=2924905745 TSecr=30102
259	75.164939	Pablos-MacBook-Air...	Pablos-MacBook-Air...	TCP	68	63621 → rfe(5002) [SYN] Seq=0 Win=65535 Len=0 MSS=16344 WS=64 TSval=366026882 TSecr=0 SACK_PERM
261	75.165154	Pablos-MacBook-Air...	Pablos-MacBook-Air...	TCP	56	63621 → rfe(5002) [ACK] Seq=1 Ack=1 Win=408320 Len=0 TSval=366026883 TSecr=577289034
262	75.165192	Pablos-MacBook-Air...	Pablos-MacBook-Air...	TCP	56	[TCP Window Update] rfe(5002) → 63621 [ACK] Seq=1 Ack=1 Win=408320 Len=0 TSval=577289035 TSecr=366026
277	82.270696	Pablos-MacBook-Air...	Pablos-MacBook-Air...	TCP	68	63622 → complex-Link(5001) [SYN] Seq=0 Win=65535 Len=0 MSS=16344 WS=64 TSval=4241314088 TSecr=0 SACK
278	82.270870	Pablos-MacBook-Air...	Pablos-MacBook-Air...	TCP	68	complex-Link(5001) → 63622 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=16344 WS=64 TSval=1393123829 T

Ilustración 4. Manejo de conexión TCP

Captura del 3-Way-Handshake (SYN, SYN-ACK, ACK) que establece el estado ESTABLISHED antes del intercambio de datos y del 4-Way-Handshake para el cierre de la conexión.

2.2. Confiabilidad y Control de Flujo

TCP garantiza la entrega confiable mediante un esquema de numeración de secuencia. En la captura, el paquete 281 muestra [PSH, ACK] Seq=1 Ack=1 Win=408320 Len=29, lo cual quiere decir que el broker envía 29 bytes de datos y el subscriber responde (paquete 282) con ACK Seq=1 Ack=30, confirmando exactamente los 29 bytes recibidos ($1 + 29 = 30$). Este ciclo se repite para cada segmento sin ninguna retransmisión en toda la captura.

No.	Time	Source	Destination	Protocol	Length	Info
278	82.270870	Pablos-MacBook-Air...	Pablos-MacBook-Air...	TCP	68	complex-link(5001) → 63622 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=16344 WS=64 TSval=1393123829
279	82.270916	Pablos-MacBook-Air...	Pablos-MacBook-Air...	TCP	56	63622 → complex-link(5001) [ACK] Seq=1 Ack=1 Win=408320 Len=0 TSval=4241314088 TSecr=1393123829
280	82.270951	Pablos-MacBook-Air...	Pablos-MacBook-Air...	TCP	56	[TCP Window Update] complex-link(5001) → 63622 [ACK] Seq=1 Ack=1 Win=408320 Len=0 TSval=1393123829
281	82.270955	Pablos-MacBook-Air...	Pablos-MacBook-Air...	TCP	85	63622 → complex-link(5001) [PSH, ACK] Seq=1 Ack=1 Win=408320 Len=29 TSval=4241314088 TSecr=1393123829
282	82.270976	Pablos-MacBook-Air...	Pablos-MacBook-Air...	TCP	56	complex-link(5001) → 63622 [ACK] Seq=1 Ack=30 Win=408320 Len=0 TSval=1393123829 TSecr=4241314088
283	82.271037	Pablos-MacBook-Air...	Pablos-MacBook-Air...	TCP	85	rfe(5002) → 63620 [PSH, ACK] Seq=1 Ack=1 Win=408320 Len=29 TSval=2924916583 TSecr=3010298339
284	82.271050	Pablos-MacBook-Air...	Pablos-MacBook-Air...	TCP	85	rfe(5002) → 63621 [PSH, ACK] Seq=1 Ack=1 Win=408320 Len=29 TSval=577296141 TSecr=366026883
285	82.271065	Pablos-MacBook-Air...	Pablos-MacBook-Air...	TCP	56	63620 → rfe(5002) [ACK] Seq=1 Ack=30 Win=408320 Len=0 TSval=3010309097 TSecr=2924916583

Ilustración 5. Confiabilidad en TCP

Captura del envío confiable de datos (29 bytes) del broker al suscriptor.

El campo Win=408320 en los paquetes del subscriber indica el tamaño de la ventana de recepción, que es el mecanismo de control de flujo. Mientras este valor sea mayor que cero, el emisor puede seguir enviando sin esperar un ACK por cada byte.

No.	Time	Source	Destination	Protocol	Length	Info
278	82.270870	Pablos-MacBook-Air...	Pablos-MacBook-Air...	TCP	68	complex-link(5001) → 63622 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=16344 WS=64 TSval=1393123829
279	82.270916	Pablos-MacBook-Air...	Pablos-MacBook-Air...	TCP	56	63622 → complex-link(5001) [ACK] Seq=1 Ack=1 Win=408320 Len=0 TSval=4241314088 TSecr=1393123829
280	82.270951	Pablos-MacBook-Air...	Pablos-MacBook-Air...	TCP	56	[TCP Window Update] complex-link(5001) → 63622 [ACK] Seq=1 Ack=1 Win=408320 Len=0 TSval=1393123829
281	82.270955	Pablos-MacBook-Air...	Pablos-MacBook-Air...	TCP	85	63622 → complex-link(5001) [PSH, ACK] Seq=1 Ack=1 Win=408320 Len=29 TSval=4241314088 TSecr=1393123829
282	82.270976	Pablos-MacBook-Air...	Pablos-MacBook-Air...	TCP	56	complex-link(5001) → 63622 [ACK] Seq=1 Ack=30 Win=408320 Len=0 TSval=1393123829 TSecr=4241314088
283	82.271037	Pablos-MacBook-Air...	Pablos-MacBook-Air...	TCP	85	rfe(5002) → 63620 [PSH, ACK] Seq=1 Ack=1 Win=408320 Len=29 TSval=2924916583 TSecr=3010298339
284	82.271050	Pablos-MacBook-Air...	Pablos-MacBook-Air...	TCP	85	rfe(5002) → 63621 [PSH, ACK] Seq=1 Ack=1 Win=408320 Len=29 TSval=577296141 TSecr=366026883
285	82.271065	Pablos-MacBook-Air...	Pablos-MacBook-Air...	TCP	56	63620 → rfe(5002) [ACK] Seq=1 Ack=30 Win=408320 Len=0 TSval=3010309097 TSecr=2924916583

Ilustración 6. Control de flujo en TCP

Captura de la capacidad de buffer del receptor para evitar saturación. En este caso es de 408.320, que representa la cantidad de bytes que el receptor tiene disponibles en su buffer para recibir datos antes de que el emisor deba detenerse y esperar una confirmación.

2.3. Orden de Entrega

Los mensajes llegan en orden estrictamente creciente. En la captura la progresión es perfectamente constante: el paquete 281 tiene Seq=1, Len=29, el paquete 289 tiene Seq=30, Len=40, el paquete 297 tiene Seq=70, Len=47, el paquete 305 tiene Seq=117, Len=54. Cada Seq de inicio es exactamente igual al Seq anterior más el Len anterior, sin ningún hueco ni salto. Esto es posible porque TCP mantiene en el kernel un buffer de reordenamiento, que quiere decir que si un segmento llega desordenado, no se entrega a la aplicación hasta que lleguen los bytes anteriores.

No.	Time	Source	Destination	Protocol	Length	Info
278	82.278878	Pablos-MacBook-Air...	Pablos-MacBook-Air...	TCP	68	complex-link(5001) → 63622 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=16344 WS=64 TSval=1393123829 TSecr=1393123829
279	82.278916	Pablos-MacBook-Air...	Pablos-MacBook-Air...	TCP	56	63622 → complex-link(5001) [ACK] Seq=1 Ack=1 Win=408320 Len=0 TSval=4241314088 TSecr=1393123829
280	82.278951	Pablos-MacBook-Air...	Pablos-MacBook-Air...	TCP	56	[TCP Window Update] complex-link(5001) → 63622 [ACK] Seq=1 Ack=1 Win=408320 Len=0 TSval=1393123829 TSecr=1393123829
281	82.278955	Pablos-MacBook-Air...	Pablos-MacBook-Air...	TCP	85	63622 → complex-link(5001) [PSH, ACK] Seq=1 Ack=1 Win=408320 Len=29 TSval=4241314088 TSecr=1393123829
282	82.278976	Pablos-MacBook-Air...	Pablos-MacBook-Air...	TCP	56	complex-link(5001) → 63622 [ACK] Seq=1 Ack=30 Win=408320 Len=0 TSval=1393123829 TSecr=4241314088
283	82.271037	Pablos-MacBook-Air...	Pablos-MacBook-Air...	TCP	85	rfe(5002) → 63620 [PSH, ACK] Seq=1 Ack=1 Win=408320 Len=29 TSval=2924916503 TSecr=3010298339
284	82.271050	Pablos-MacBook-Air...	Pablos-MacBook-Air...	TCP	85	rfe(5002) → 63621 [PSH, ACK] Seq=1 Ack=1 Win=408320 Len=29 TSval=577296141 TSecr=366026883
285	82.271065	Pablos-MacBook-Air...	Pablos-MacBook-Air...	TCP	56	63620 → rfe(5002) [ACK] Seq=1 Ack=30 Win=408320 Len=0 TSval=3010309097 TSecr=2924916503
286	82.271073	Pablos-MacBook-Air...	Pablos-MacBook-Air...	TCP	56	63621 → rfe(5002) [ACK] Seq=1 Ack=30 Win=408320 Len=0 TSval=366033989 TSecr=577296141
289	83.276007	Pablos-MacBook-Air...	Pablos-MacBook-Air...	TCP	96	63622 → complex-link(5001) [PSH, ACK] Seq=30 Ack=1 Win=408320 Len=40 TSval=4241315093 TSecr=13931238
290	83.276081	Pablos-MacBook-Air...	Pablos-MacBook-Air...	TCP	56	complex-link(5001) → 63622 [ACK] Seq=1 Ack=70 Win=408320 Len=0 TSval=1393124835 TSecr=4241315093
291	83.276133	Pablos-MacBook-Air...	Pablos-MacBook-Air...	TCP	96	rfe(5002) → 63620 [PSH, ACK] Seq=30 Ack=1 Win=408320 Len=40 TSval=2924917508 TSecr=3010309097
292	83.276139	Pablos-MacBook-Air...	Pablos-MacBook-Air...	TCP	96	rfe(5002) → 63621 [PSH, ACK] Seq=30 Ack=1 Win=408320 Len=40 TSval=577297146 TSecr=366033989
293	83.276169	Pablos-MacBook-Air...	Pablos-MacBook-Air...	TCP	56	63620 → rfe(5002) [ACK] Seq=1 Ack=70 Win=408320 Len=0 TSval=3010310102 TSecr=2924917508
294	83.276185	Pablos-MacBook-Air...	Pablos-MacBook-Air...	TCP	56	63621 → rfe(5002) [ACK] Seq=1 Ack=70 Win=408320 Len=0 TSval=366034994 TSecr=577297146
297	84.281183	Pablos-MacBook-Air...	Pablos-MacBook-Air...	TCP	103	63622 → complex-link(5001) [PSH, ACK] Seq=70 Ack=1 Win=408320 Len=47 TSval=4241316099 TSecr=13931248
298	84.281274	Pablos-MacBook-Air...	Pablos-MacBook-Air...	TCP	56	complex-link(5001) → 63622 [ACK] Seq=1 Ack=117 Win=408320 Len=0 TSval=1393125840 TSecr=4241316099
299	84.281411	Pablos-MacBook-Air...	Pablos-MacBook-Air...	TCP	103	rfe(5002) → 63620 [PSH, ACK] Seq=70 Ack=1 Win=408320 Len=47 TSval=2924918513 TSecr=3010310102
300	84.281455	Pablos-MacBook-Air...	Pablos-MacBook-Air...	TCP	56	63620 → rfe(5002) [ACK] Seq=1 Ack=117 Win=408320 Len=0 TSval=3010311107 TSecr=2924918513
301	84.281470	Pablos-MacBook-Air...	Pablos-MacBook-Air...	TCP	103	rfe(5002) → 63621 [PSH, ACK] Seq=70 Ack=1 Win=408320 Len=47 TSval=577298151 TSecr=366034994
302	84.281491	Pablos-MacBook-Air...	Pablos-MacBook-Air...	TCP	56	63621 → rfe(5002) [ACK] Seq=1 Ack=117 Win=408320 Len=0 TSval=366035999 TSecr=577298151
305	85.285242	Pablos-MacBook-Air...	Pablos-MacBook-Air...	TCP	110	63622 → complex-link(5001) [PSH, ACK] Seq=117 Ack=1 Win=408320 Len=54 TSval=4241317103 TSecr=1393125

Ilustración 7. Orden de entrega en TCP

En la captura se observa la secuencia lógica de números de seguimiento que garantizan que el flujo de bytes se reconstruya en el destino sin desorden

2.4. Pérdida de Mensajes

No hay ninguna pérdida. En la captura no aparece ningún paquete con la etiqueta [TCP Retransmission], [TCP Fast Retransmission] ni [TCP Dup ACK], que son los marcadores que Wireshark colorea en rojo/negro cuando detecta pérdida. Dado ese caso, el receptor no enviaría el ACK correspondiente, obligando al emisor a retransmitir el segmento tras un timeout, garantizando que ningún mensaje se pierda en el camino.

Time	Pablos-MacBook-Air.local	Pablos-MacBook-Air.local	Comment
38.743195	5002	rfe(5002) → 63546 [FIN, ACK] Seq=1 Ack=1 Win=...	TCP: rfe(5002) → 63546 [FIN, ACK] Seq=1 Ack=1.
38.743220	5002	rfe(5002) → 63544 [FIN, ACK] Seq=1 Ack=1 Win=...	TCP: rfe(5002) → 63544 [FIN, ACK] Seq=1 Ack=1.
38.743275	5002	63546 → rfe(5002) [ACK] Seq=1 Ack=2 Win=63...	TCP: 63546 → rfe(5002) [ACK] Seq=1 Ack=2 Win...
38.743295	5002	63544 → rfe(5002) [ACK] Seq=1 Ack=2 Win=63...	TCP: 63544 → rfe(5002) [ACK] Seq=1 Ack=2 Win...
38.744064	5002	63546 → rfe(5002) [FIN, ACK] Seq=1 Ack=2 Wi...	TCP: 63546 → rfe(5002) [FIN, ACK] Seq=1 Ack=2.
38.744117	5002	rfe(5002) → 63546 [ACK] Seq=2 Ack=2 Win=63...	TCP: rfe(5002) → 63546 [ACK] Seq=2 Ack=2 Wi...
38.744401	5002	63544 → rfe(5002) [FIN, ACK] Seq=1 Ack=2 Wi...	TCP: 63544 → rfe(5002) [FIN, ACK] Seq=1 Ack=2.
38.744440	5002	rfe(5002) → 63544 [ACK] Seq=2 Ack=2 Win=63...	TCP: rfe(5002) → 63544 [ACK] Seq=2 Ack=2 Wi...
71.513799	5002	63620 → rfe(5002) [SYN] Seq=0 Win=65535 Le...	TCP: 63620 → rfe(5002) [SYN] Seq=0 Win=6553.
71.513881	5002	rfe(5002) → 63620 [SYN, ACK] Seq=0 Ack=1 Wi...	TCP: rfe(5002) → 63620 [SYN, ACK] Seq=0 Ack=...
71.513901	5002	63620 → rfe(5002) [ACK] Seq=1 Ack=1 Win=40...	TCP: 63620 → rfe(5002) [ACK] Seq=1 Ack=1 Win...
71.513914	5002	[TCP Window Update] rfe(5002) → 63620 [ACK]	TCP: [TCP Window Update] rfe(5002) → 63620 [...]
75.164939	5002	63621 → rfe(5002) [SYN] Seq=0 Win=65535 Le...	TCP: 63621 → rfe(5002) [SYN] Seq=0 Win=6553.
75.165110	5002	rfe(5002) → 63621 [SYN, ACK] Seq=0 Ack=1 Wi...	TCP: rfe(5002) → 63621 [SYN, ACK] Seq=0 Ack=...
75.165154	5002	63621 → rfe(5002) [ACK] Seq=1 Ack=1 Win=40...	TCP: 63621 → rfe(5002) [ACK] Seq=1 Ack=1 Win...
75.165192	5002	[TCP Window Update] rfe(5002) → 63621 [ACK]	TCP: [TCP Window Update] rfe(5002) → 63621 [...]
82.270696	5001	63622 → complex-link(5001) [SYN] Seq=0 Wi...	TCP: 63622 → complex-link(5001) [SYN] Seq=...
82.270870	5001	complex-link(5001) → 63622 [SYN, ACK] Seq...	TCP: complex-link(5001) → 63622 [SYN, ACK] ...
82.270916	5001	63622 → complex-link(5001) [ACK] Seq=1 Ack...	TCP: 63622 → complex-link(5001) [ACK] Seq=...
82.270951	5001	[TCP Window Update] complex-link(5001) → 6...	TCP: [TCP Window Update] complex-link(5001).
82.270955	5001	63622 → complex-link(5001) [PSH, ACK] Seq...	TCP: 63622 → complex-link(5001) [PSH, ACK] ...

Ilustración 8. Diagrama de flujo temporal de la red en TCP

En la captura se observa el flujo continuo de paquetes (sin huecos) en la numeración de secuencia, confirmando la entrega completa de mensajes sin pérdida

2.5. Overhead de Cabeceras/Protocolo

TCP presenta un overhead significativo en comparación con UDP. En la captura, se observa una longitud de cabecera (Header Length) de 32 bytes. Para un mensaje de 54 bytes (payload), la cabecera representa aproximadamente el 37% del segmento total en la capa de

transporte. Este tamaño extendido, que supera los 20 bytes base de TCP, se debe a la inclusión de opciones como Timestamps (10 bytes) y No-Operation (NOP), herramientas fundamentales para el cálculo del RTT y la optimización de la conexión.

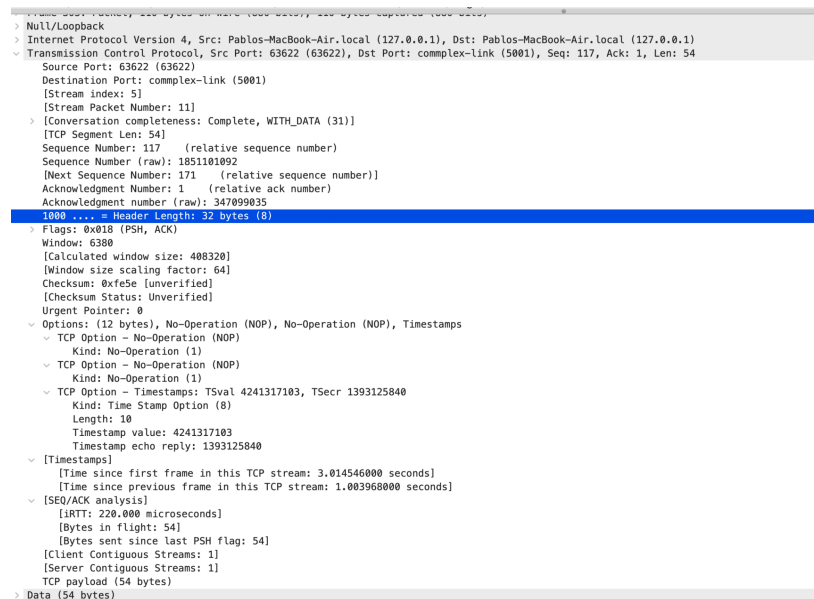


Ilustración 9. Overhead de cabeceras/protocolo en TCP

En la captura se observa una cabecera de 32 bytes, evidenciando el uso de opciones adicionales para la sincronización temporal, lo que incrementa el overhead del protocolo en comparación con soluciones no orientadas a la conexión.

3. Tabla comparativa:

	TCP	UDP
Confiabilidad	Alta, garantizada mediante el uso de acuses de recibo (ACK) y retransmisiones automáticas de segmentos perdidos.	Nula, modelo fire-and-forget. Si un datagrama se pierde o corrompe en la red, no es retransmitido.
Orden de entrega	Garantizado, utiliza números de secuencia para asegurar que la aplicación reciba los datos en el orden exacto en que fueron enviados.	No garantizado, los datagramas pueden tomar distintas rutas y llegar desordenados, udp los entrega a la aplicación tal como llegan
Pérdida de mensajes	Mínima, detecta paquetes faltantes y los reenvía antes de entregar la	Frecuentemente bajo congestión, si los buffers de

	información, evitando "huecos" en el flujo de datos.	los routers o del receptor se llenan, los mensajes se descartan silenciosamente.
Overhead de cabeceras/protocolo	Significativo, con una cabecera mínima de 20 bytes que puede aumentar hasta 60 bytes al incluir opciones de control y timestamps.	Muy abajo porque la cabecera tiene un tamaño fijo de 8 bytes(puertos, longitud y checksum) reduciendo el consumo de ancho de banda.

4. Análisis de las Preguntas Planteadas

1. **¿Qué ocurriría si en lugar de dos publicadores (partidos transmitidos) hubiera cien partidos simultáneos? ¿Cómo impactaría esto en el desempeño del broker bajo TCP y bajo UDP?**

Con cien partidos simultáneos, el desempeño del broker bajo TCP se vería comprometido por el alto consumo de CPU y memoria, ya que el sistema operativo debe mantener una tabla de estados y buffers individuales para cada una de las cientos de conexiones activas. En contraste, bajo UDP el impacto sería significativamente menor debido a que es un protocolo no orientado a la conexión. El broker simplemente procesa datagramas de forma independiente a medida que llegan, eliminando el overhead de gestionar handshakes y estados de sesión persistentes para cada publicador.

2. **Si un gol se envía como mensaje desde el publicador y un suscriptor no lo recibe en UDP, ¿qué implicaciones tendría para la aplicación real? ¿Por qué TCP maneja mejor este escenario?**

Si un mensaje de gol se pierde en UDP, la implicación para la aplicación real es una inconsistencia crítica en la información, donde el usuario vería un marcador desactualizado de forma indefinida al no existir un mecanismo de retransmisión. TCP maneja mejor este escenario porque su arquitectura de confiabilidad utiliza mensajes de recibo (ACKs) y temporizadores. Esto quiere decir que si el publicador no recibe la confirmación del broker, reenvía el paquete automáticamente, asegurando que cada evento vital del partido llegue finalmente al suscriptor.

3. **En un escenario de seguimiento en vivo de partidos, ¿qué protocolo (TCP o UDP) resultaría más adecuado? Justifique con base en los resultados de la práctica.**

Con base en los resultados de la práctica, TCP resulta más adecuado para un sistema de seguimiento deportivo en tiempo real donde la integridad de la información es crítica. Las capturas demostraron que TCP entregó los mensajes completos, en orden estrictamente creciente y sin ninguna retransmisión, garantizando que cada suscriptor recibiera exactamente el mismo resultado. Eventos como goles, tarjetas o el resultado final no admiten pérdida ni reordenamiento, ya que un marcador desordenado o incompleto genera una experiencia incorrecta para el usuario. UDP sería preferible únicamente si la aplicación priorizara latencia mínima sobre exactitud, por ejemplo, streaming de video o audio donde un frame perdido es aceptable, pero en un sistema de actualización de eventos discretos e irrepetibles como los de un partido de fútbol, la confiabilidad y el orden que ofrece TCP son indispensables.

4. Compare el overhead observado en las capturas Wireshark entre TCP y UDP. ¿Cuál protocolo introduce más cabeceras por mensaje? ¿Cómo influye esto en la eficiencia?

TCP introduce significativamente más cabeceras por mensaje. Mientras que la cabecera de UDP observada tiene un tamaño fijo y mínimo de apenas 8 bytes, la cabecera de TCP es de al menos 20 bytes (y puede ser mayor si incluye opciones). TCP necesita este espacio extra para transportar los números de secuencia, números de acuse de recibo (ACKs), tamaño de ventana y las banderas de control (SYN, ACK, FIN, etc.) necesarios para garantizar la confiabilidad. Esta diferencia impacta directamente en la eficiencia de la red. UDP es mucho más eficiente en términos de aprovechamiento del ancho de banda y velocidad de procesamiento. Al tener un overhead (carga útil) menor, un mayor porcentaje de cada trama de red transmitida corresponde a datos reales de la aplicación (payload). Además, al no tener que procesar el estado de la conexión ni esperar ACKs, los routers y el sistema operativo pueden despachar y enrutar datagramas UDP a una velocidad considerablemente mayor, lo que lo hace ideal para sistemas de transmisión en tiempo real donde la velocidad es más crítica que la entrega perfecta.

5. Si el marcador de un partido llega desordenado en UDP (por ejemplo, primero se recibe el 2-1 y luego el 1-1), ¿qué efectos tendría en la experiencia del usuario? ¿Cómo podría solucionarse este problema a nivel de aplicación?

Si los datagramas llegan desordenados en una aplicación de seguimiento deportivo, un usuario vería que su equipo pasa de ir ganando 2-1 a, de repente, estar empatando 1-1 en el marcador en vivo. Esta inconsistencia temporal anula el propósito fundamental de una aplicación de noticias en tiempo real. Dado que UDP no garantiza el ordenamiento nativo en la capa de transporte, la solución debe implementarse estrictamente en la capa de aplicación. Se puede lograr inyectando metadatos en el payload del mensaje.

Ejemplo: uso de Timestamps o Sequence Numbers, el publicador debe incluir una marca de tiempo (timestamp) o un número de secuencia incremental dentro de la cadena de texto de cada mensaje (ej. SEQ:45 Gol de Equipo A). Luego el Suscriptor debe llevar un registro del último número de secuencia válida recibido. Si llega un datagrama con un número de secuencia menor o una marca de tiempo más antigua que el estado actual que se muestra en

pantalla, la aplicación simplemente descarta ese paquete "viejo" o desordenado, asegurando que la interfaz del usuario solo avance hacia adelante en el tiempo.

6. ¿Cómo cambia el desempeño del sistema cuando aumenta el número de suscriptores interesados en un mismo partido? ¿Qué diferencias se observaron entre TCP y UDP en este aspecto?

A medida que el número de suscriptores aumenta a miles o millones, el sistema experimenta un cuello de botella en el nodo central (Broker). El Broker debe iterar sobre una lista cada vez más grande de clientes, replicando y enviando el mismo mensaje a cada uno de forma individual. Esto genera un consumo linealmente creciente de CPU y ancho de banda en el servidor, degradando el tiempo que tarda el último suscriptor de la lista en recibir la actualización comparado con el primero.

En TCP la degradación del desempeño es exponencial y crítica. Como TCP está orientado a la conexión, el Broker se ve obligado a mantener un socket abierto, variables de estado, y buffers de memoria independientes para cada suscriptor. Además, si la red de algunos clientes es lenta, el control de flujo de TCP puede bloquear los hilos del Broker mientras espera los ACKs, causando retrasos masivos o incluso colapsando el servidor por falta de memoria RAM.

En UDP la escala es mucho más eficiente bajo esta arquitectura. Al ser stateless, el Broker no guarda estado de conexiones ni mantiene buffers por cliente. Solo almacena un arreglo de direcciones IP y dispara los datagramas a máxima velocidad. El Broker UDP no se bloquea esperando confirmaciones, permitiéndole atender a una cantidad significativamente mayor de suscriptores con los mismos recursos de hardware.

7. ¿Qué sucede si el broker se detiene inesperadamente? ¿Qué diferencias hay entre TCP y UDP en la capacidad de recuperación de la sesión?

En TCP, la caída del broker es detectada casi de inmediato por todos los clientes conectados. Dado que TCP mantiene conexiones persistentes mediante el handshake, cuando el broker cae, el sistema operativo envía un paquete RST o simplemente deja de responder, lo que provoca que cada publisher y subscriber reciba un error de conexión. Esto significa que todas las sesiones activas se rompen simultáneamente y ningún cliente puede seguir operando sin reconectarse explícitamente, lo que involucra que para restaurar el servicio es necesario que se vuelva a hacer la conexión (con Handshake y todo) de parte de todos los elementos del servicio.

En UDP, el broker no mantiene ningún estado de conexión con publishers ni subscribers. Si el broker cae, los publishers simplemente siguen enviando datagramas al puerto del broker sin recibir ningún error inmediato (UDP no tiene mecanismo de retroalimentación), y los suscriptores dejan de recibir mensajes silenciosamente, sin ninguna notificación de fallo. La recuperación en UDP es más sencilla: cuando el broker vuelve a levantarse, los publishers pueden reanudar el envío de datagramas de inmediato sin necesidad de restablecer nada. Sin

embargo, todos los mensajes enviados durante la caída se pierden definitivamente, sin posibilidad de retransmisión.

En resumen, TCP falla de forma notable y detectable de tal manera que todos se dan cuenta de esto; en cambio UDP falla silenciosamente.

8. ¿Cómo garantizar que todos los suscriptores reciban en el mismo instante las actualizaciones críticas (por ejemplo, un gol)? ¿Qué protocolo facilita mejor esta sincronización y por qué?

La sincronización perfecta entre múltiples suscriptores es uno de los retos más difíciles en sistemas distribuidos, se tiene que:

Cada tipo de protocolo de transporte maneja esto de manera única, en el caso del TCP se tiene que tiene conexiones directas con los usuarios por lo que todos tienen latencias diversas y por lo mismo se manejan tiempos únicos pero se va enviando lo mismo al mismo tiempo y solo se generan problemas cuando hay un retraso o pérdida de paquetes aunque esto no significa que los suscriptores reciban la información a destiempo sino que es más propenso a recibirla más tarde pero garantiza recibirla y en perfecto estado. En el caso del UDP se tiene que al no tener que recibir una retroalimentación ni revisar nada este manda todo de manera simultánea y con muy baja latencia lo cual cumple mejor el elemento de actualizaciones de manera instantánea pero no asegura la integridad de esta información.

Por tanto, UDP facilita mejor la sincronización en términos de arquitectura, especialmente con multicast. TCP ofrece garantía de entrega pero sacrifica simultaneidad cuando el número de suscriptores crece.

9. Analice el uso de CPU y memoria en el broker cuando maneja múltiples conexiones TCP frente al manejo de datagramas UDP. ¿Qué diferencias encontró?

Esta diferencia es una de las más relevantes al escalar el sistema.

En TCP, el broker mantiene un file descriptor abierto por cada cliente conectado (publisher o subscriber). En la implementación del laboratorio con hilos por conexión, cada conexión consume:

- Un socket con sus buffers de envío y recepción en el kernel
- Si se usaron hilos los stacks de estos.
- La CPU debe gestionar el estado de cada conexión, los ACKs, el control de flujo y la detección de desconexiones.

Con dos publishers y dos subscribers, el broker TCP maneja cuatro conexiones simultáneas manejables estables y funcionales. Pero el modelo escala con 100 suscriptores implican 100 sockets abiertos, 100 buffers en kernel y potencialmente 100 hilos lo cual genera un gran

consumo de recursos y por lo mismo es muy exigente con la CPU que corre el servicio y por lo mismo es poco escalable o más costoso.

En UDP, el broker usa un único socket para recibir todos los datagramas de todos los publishers, y otro para reenviar a los suscriptores. No hay estado persistente por cliente, o sea el broker simplemente lee datagramas con (qué incluye la dirección del remitente) y hace la envía a cada suscriptor registrado. El consumo de memoria es prácticamente constante independientemente del número de clientes, y la CPU solo procesa paquetes cuando llegan, sin mantenimiento de conexiones.

O sea como se ha visto en todo el laboratorio el uso de TCP trae más ventajas a nivel práctico pero involucra muchas complejidades, entre las más importantes es el uso tan grande de recursos de CPU que tiene la contraposición con el UDP.

10. Si tuviera que diseñar un sistema real de transmisión de actualizaciones de partidos de fútbol para millones de usuarios, ¿elegiría TCP, UDP o una combinación de ambos? Justifique con base en lo observado en el laboratorio

Para un sistema real de transmisión de actualizaciones de partidos a escala masiva la mejor estrategia es implementar una combinación estratégica:

UDP (con QUIC) para la distribución masiva de actualizaciones. Como se vio, el broker UDP escala mucho mejor en memoria y CPU, y la entrega es más simultánea. Los mensajes de estado de un partido (marcador, eventos) son tolerantes a pérdidas ocasionales si se implementa un número de secuencia a nivel de aplicación el suscriptor puede detectar que se saltó un evento y solicitarlo por separado.

TCP para operaciones críticas de control el registro de suscriptores al broker, la autenticación, y la entrega garantizada de eventos de alta importancia (gol, resultado final) donde la pérdida es inaceptable. También es preferible para clientes en redes con alta pérdida de paquetes donde UDP sería demasiado poco confiable.

En conclusión, lo que planteamos y consideramos que es la mejor estrategia para este caso es coger las ventajas de los dos mundos y aprovecharlas, usar la baja latencia y eficacia del UDP para la implementación mayoritaria del servicio pero coger las características de recuperación y confiabilidad de los datos de TCP para generar las primeras conexiones y para los eventos más importantes que no se pueden arriesgar a perder debido a su importancia.